

AI Lab # 3

This lab explores uninformed search strategies by implementing Depth-First Search and Uniform Cost Search to systematically find solutions in state-space problems.

Artificial Intelligence (LAB)

Instructor: Mubbashir Ahmed

Email: mubbashir.ahmed@iqra.edu.pk

Date: 02 August, 2026

Today's Agenda

01

What is DFS?

02

Algorithm Steps and Example

03

What is UCS?

04

Algorithm Steps and Example

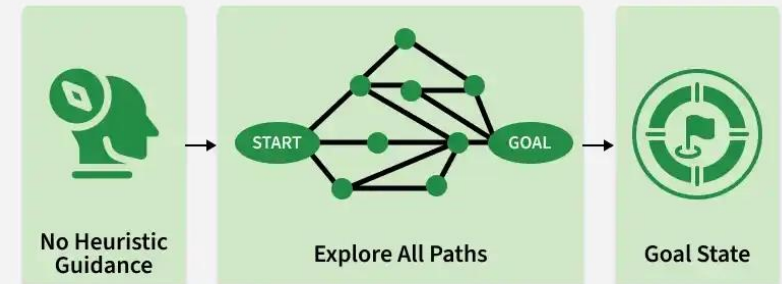
05

Lab Tasks

Uninformed Search

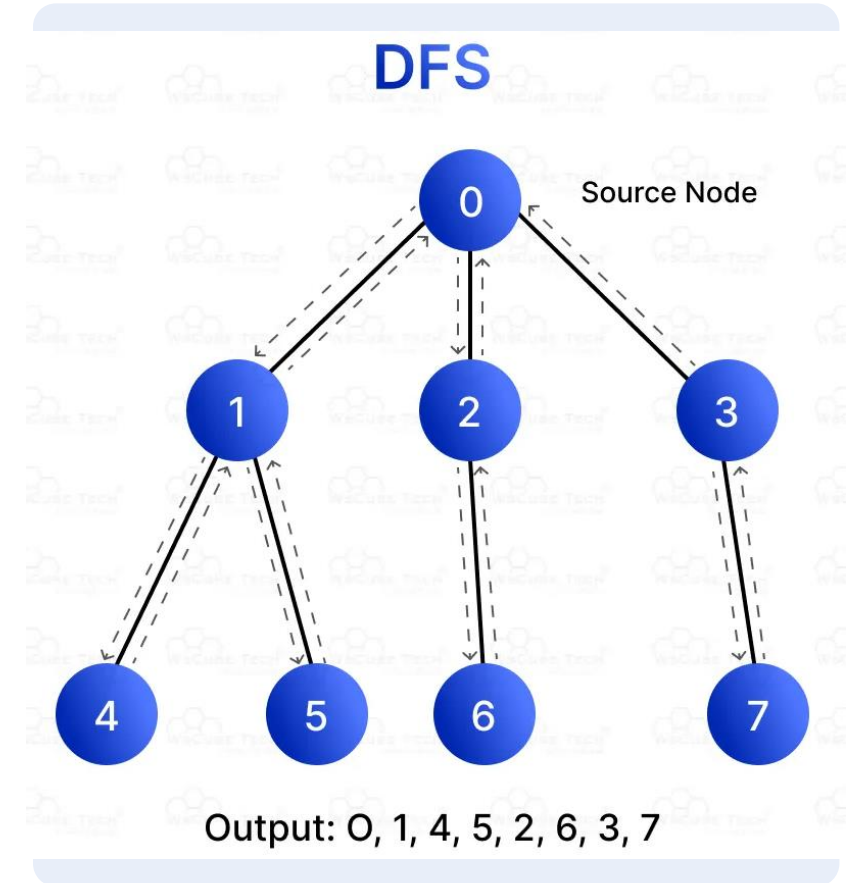
- Uninformed search algorithms also called blind search algorithms.
- They have no additional information about goal location except the problem definition.
- They explore the search space blindly.
- Examples are BFS, DFS, UCS

Uninformed Search in Artificial Intelligence



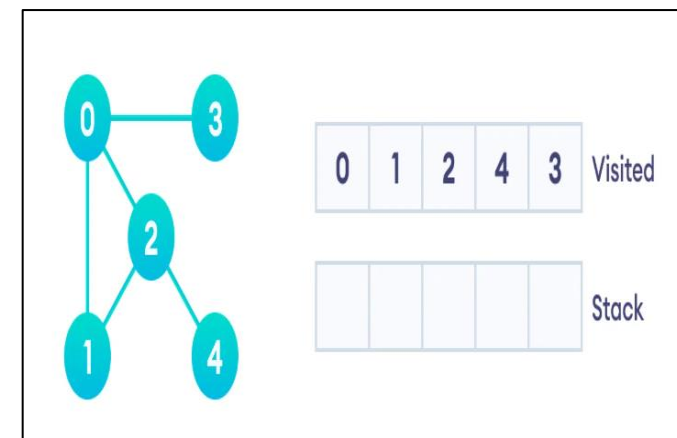
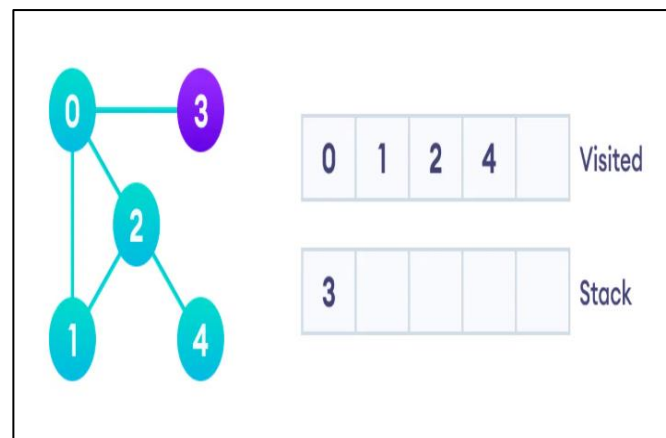
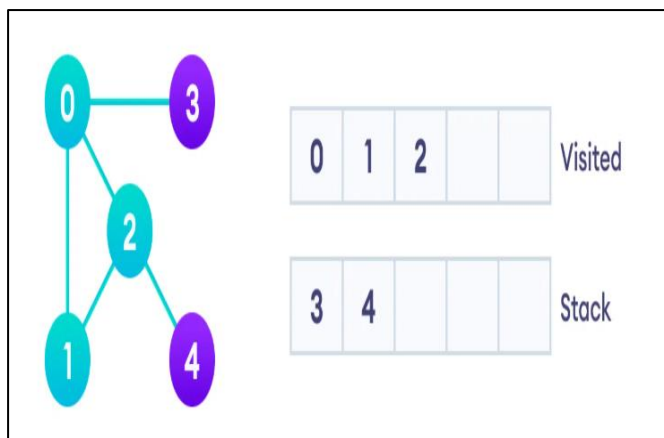
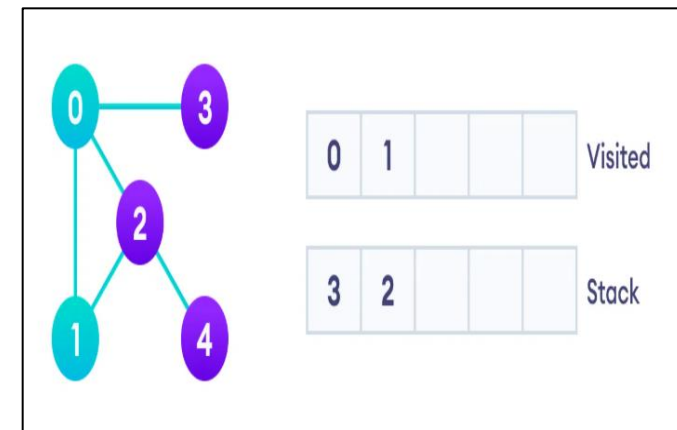
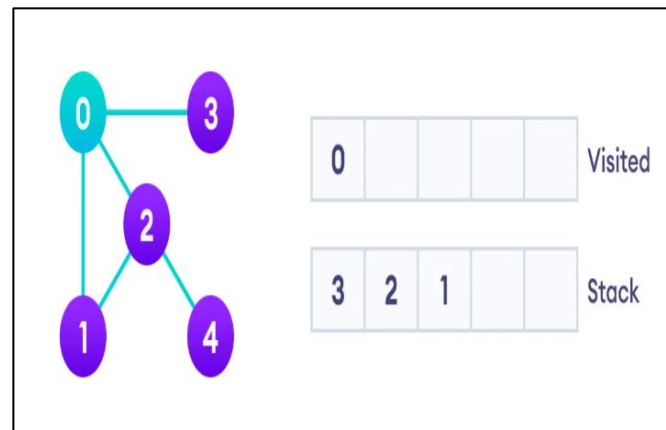
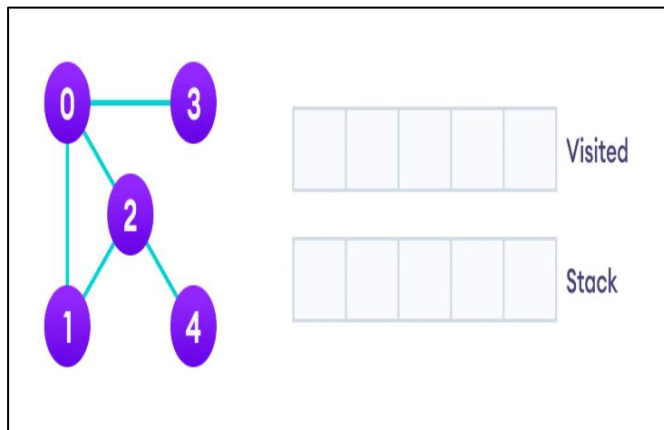
What is DFS?

- DFS is an uninformed search algorithm.
- Explores nodes by going deep as possible along one branch before backtracking.
- All nodes in the current branch are visited before exploring the next branch.



Algorithm Steps

- 1** **Push the start node:** Put starting node onto the stack
- 2** **Pop and visit:** Take the front item from the stack and add it into the visited list
- 3** **Push unvisited neighbors:** Find adjacent nodes that are not visited and add them onto the stack
- 4** **Repeat until the stack is empty:** Keep repeating step 2 and 3 until the stack is empty



Example of DFS

Now that we've covered DFS, let's move towards practical code-based example

DFS

Depth First Search on nodes 0, 1, 2, 3, 4. Starting node is 0.



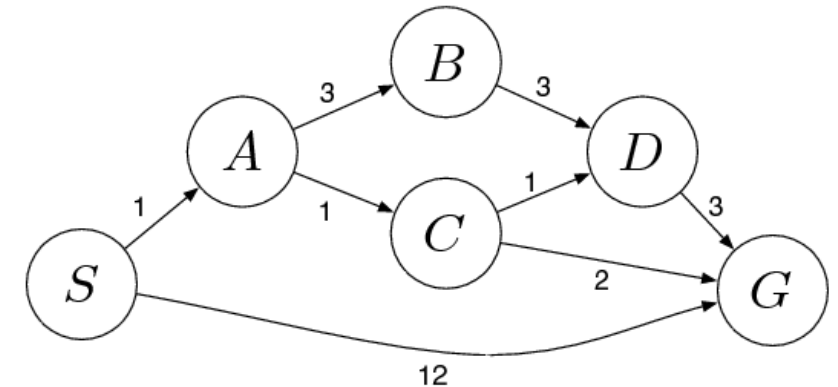
```
graph = { 0: [1, 2, 3], 1: [0, 2], 2: [0, 1, 4], 3: [0], 4: [2] }
```

```
def dfs(graph, start):  
    visited = []  
    stack = [start]  
    while stack:  
        node = stack.pop()  
        if node not in visited:  
            visited.append(node)  
            stack.extend(reversed(graph[node]))  
  
    return visited  
  
result = dfs(graph, 0)  
print("DFS Traversal:", result)
```

What is UCS?

- UCS is an uninformed search algorithm.
- Explores nodes based on the lowest total cost of path, not by level or depth.
- Node with the lowest total cost is always expanded before any other node in the queue.

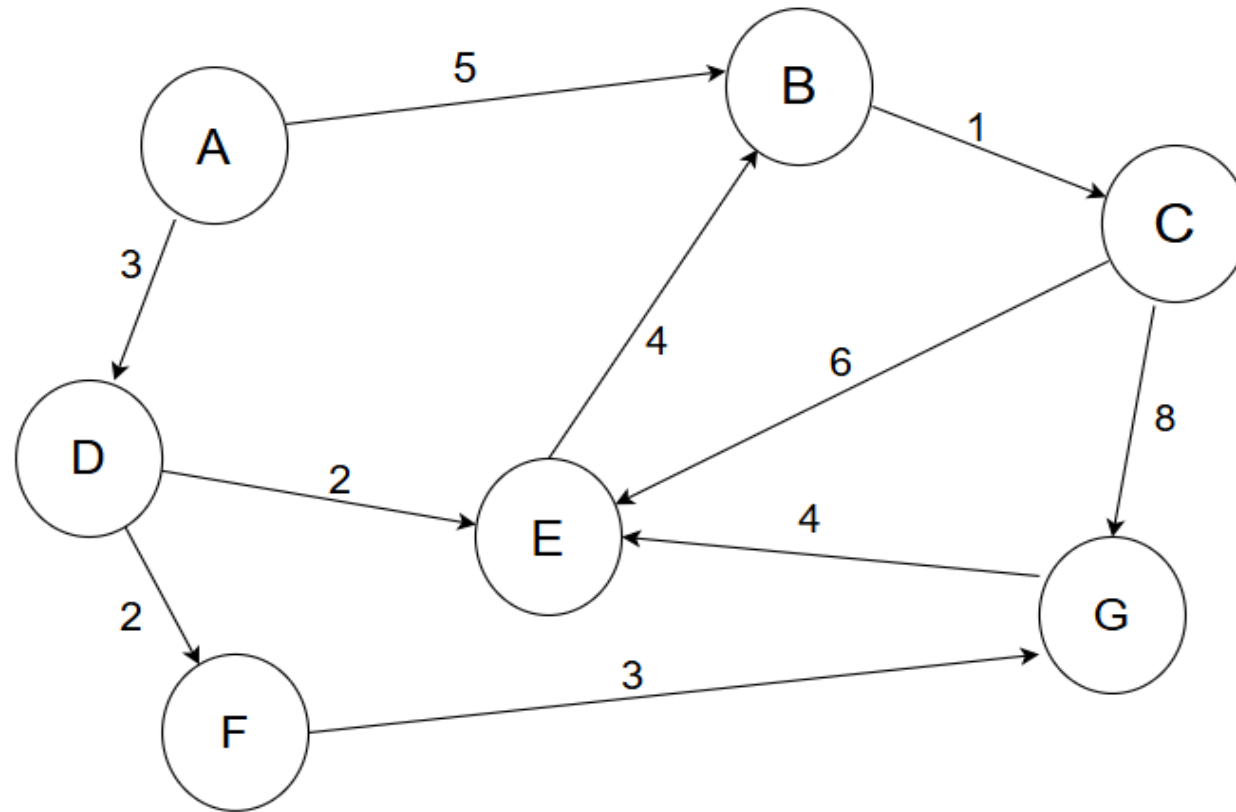
Priority queue is a special kind of list where items with the smallest number comes out first



Algorithm Steps

- 1 Add the start node:** Put starting node in the priority queue with cost 0
- 2 Remove the lowest-cost path:** Take the path with the lowest total cost from the priority queue
- 3 Check for goal:** If this node is the goal, stop and return the path and total cost
- 4 Expand neighbors:** If not the goal, look at its neighboring nodes, calculate their new total cost, and add or update them in the priority queue if this path is cheaper
- 5 Repeat until the goal is found or queue is empty:** Keep repeating step 2 to 4 until the goal is found or the priority queue is empty

Consider the following graph. Let the starting node be A and the goal node be G.



Finding path A to G using uniform cost search

Step	Frontier List
1	{{(A, 0)}}
2	{{(A-D, 3), (A-B, 5)}}
3	{{(A-B, 5), (A-D-E, 5), (A-D-F, 5)}}
4	{{(A-D-E, 5), (A-D-F, 5), (A-B-C, 6)}}
5	{{(A-D-F, 5), (A-B-C, 6), (A-D-E-B, 9)}}
6	{{(A-B-C, 6), (A-D-F-G, 8)}}
7	{{(A-D-F-G, 8), (A-B-C-E, 12), (A-B-C-G, 14)}}
8	{{(A-D-F-G, 8)}}

Example of UCS

Now that we've covered UCS, let's move towards practical code-based example

Uniform Cost Search on nodes A, B, C, D, E, F, G. Starting node is A.



```
import heapq
graph = { 'A': [('B', 5), ('D', 3)], 'B': [('C', 1)], 'C': [('E', 6), ('G', 8)], 'D': [('E', 2), ('F', 2)], 'E': [('B', 4)], 'F': [('G', 3)], 'G':
[] }

def ucs(graph, start, goal):
    visited = set()
    queue = [(0, start, [start])]
    while queue:
        cost, node, path = heapq.heappop(queue)
        if node == goal:
            return path, cost
        if node in visited:
            continue
        visited.add(node)
        for neighbor, step in graph[node]:
            if neighbor not in visited:
                heapq.heappush(queue, (cost + step, neighbor, path + [neighbor]))
    return None

path, cost = ucs(graph, 'A', 'G')
print("UCS Path:", path, "with Cost:", cost)
```

Lab Task: Depth First Search and Uniform Cost Search (Uninformed Search)

- Write a DFS program to find a path in a given graph and display the visited nodes.

Given Graph:

$A \rightarrow B, C$

$B \rightarrow D, E$

$C \rightarrow F$

$D \rightarrow G$

$E \rightarrow G$

$F \rightarrow G$

- Write a UCS program to find the cheapest path in a weighted graph and show its path and total cost.

Given Graph:

$A \text{ --2--} \rightarrow B$

$A \text{ --5--} \rightarrow C$

$B \text{ --2--} \rightarrow D$

$B \text{ --7--} \rightarrow C$

$C \text{ --1--} \rightarrow D$

$D \text{ --3--} \rightarrow G$

$C \text{ --6--} \rightarrow G$

Thank You

Questions or discussions are welcomed!